Relatório Arquitetural: Plataforma EventHub

Projeto e Implementação de um Sistema Baseado em Microsserviços

1st Vitor Fontenele
State University of Ceará
vitor.fontenele@aluno.uece.br

2nd Ian Domingos
State University of Ceará
ian.domingos@aluno.uece.br

3nd Marx Haron G. Barbosa
State University of Ceará
marx.haron@aluno.uece.br

4nd Lucas Aguiar
State University of Ceará
lucas.aguiar@aluno.uece.br

5nd Marcos Aurélio
State University of Ceará
mar.aurelio@aluno.uece.br

***Resumo*—Este relatório técnico apresenta o projeto arquitetural e a materialização em protótipo executável do EventHub, uma plataforma para gerenciamento de eventos acadêmicos e profissionais. O documento detalha a análise do domínio, a estratégia de decomposição em microsserviços autônomos, as decisões de persistência e comunicação híbrida (síncrona e assíncrona), além de apresentar evidências experimentais de controle de concorrência e resiliência a falhas.**

I. Introdução

Sistemas voltados à gestão de eventos acadêmicos e profissionais, como congressos, simpósios e minicursos, apresentam variações significativas de demanda ao longo de seu funcionamento. Embora atividades como planejamento e gerenciamento da programação possuam uma carga relativamente previsível, a abertura de inscrições pode gerar picos de acessos simultâneos, especialmente quando há vagas limitadas. Em arquiteturas monolíticas, esse cenário dificulta o escalonamento seletivo e aumenta o impacto de falhas em serviços externos, podendo comprometer operações essenciais.

Nesse contexto, o EventHub foi concebido como uma plataforma multi-organização para centralizar o gerenciamento de eventos, desde sua criação até o acompanhamento dos participantes. A solução permite que organizadores administrem eventos, atividades, inscrições, credenciamento e indicadores, enquanto participantes podem consultar programações, acompanhar a disponibilidade de vagas, solicitar inscrições, realizar pagamentos e emitir certificados.

Este trabalho tem como objetivo projetar e implementar uma arquitetura distribuída baseada em microsserviços para o EventHub, buscando estabelecer serviços com alta coesão e baixo acoplamento, definir estratégias de comunicação e propriedade dos dados e garantir a integridade das operações críticas. Entre os principais desafios estão a prevenção de superlotação (*overbooking*), o controle de inscrições concorrentes e o gerenciamento de listas de espera.

Para validar a proposta, será desenvolvido um protótipo integrado contemplando autenticação e controle de acesso, gestão de eventos e atividades, catálogo público, alocação de inscrições, processamento financeiro e notificações assíncronas. Esses componentes serão integrados por meio de uma API Gateway e disponibilizados através de uma interface web responsiva. Além disso, serão realizados testes de concorrência e injeção de falhas para avaliar a escalabilidade, a tolerância a falhas e as complexidades operacionais decorrentes da distribuição do sistema.

II. Análise do Domínio e Requisitos

A. Domínio de Negócio e Requisitos Funcionais

O EventHub atua no gerenciamento de eventos acadêmicos e corporativos em um modelo multi-organização. Cada organização pode criar eventos e estruturar sua programação em atividades, que possuem informações como local, horário, responsável, capacidade, valor de inscrição e política de lista de espera.

O sistema atende a dois perfis principais: organizadores e participantes. Os organizadores são responsáveis pela criação, edição e publicação de eventos, gerenciamento de atividades, acompanhamento de inscrições, controle de presença e emissão de certificados. Já os participantes podem consultar eventos publicados, verificar a disponibilidade de vagas, realizar inscrições e pagamentos, acompanhar notificações e obter certificados de participação.

Para atender a esses fluxos, o sistema contempla funcionalidades de cadastro, autenticação e controle de permissões; gerenciamento de eventos e atividades; busca pública com filtros por instituição, gratuidade e disponibilidade de vagas; gerenciamento de inscrições e listas de espera; processamento simulado de pagamentos, incluindo aprovação, recusa e expiração; envio de notificações; registro de presença por meio de *check-in*; emissão de certificados verificáveis; e geração de relatórios com indicadores de inscrições, desistências, presenças e receitas.

B. Regras de Negócio

O funcionamento do EventHub depende de regras que garantem a integridade das operações. A principal delas é o controle de capacidade, que impede inscrições acima do limite de vagas, mesmo diante de requisições simultâneas. Quando não houver disponibilidade, os participantes devem ser direcionados à lista de espera, caso esteja habilitada, respeitando a ordem de chegada (FIFO).

Nas atividades pagas, a inscrição gera uma reserva provisória, com prazo de validade, e sua confirmação depende exclusivamente da aprovação do pagamento. Em caso de

recusa, expiração ou cancelamento, a vaga deve ser liberada para outro participante, seguindo a política da lista de espera.

Outras regras envolvem o desacoplamento das notificações, cujas falhas não podem comprometer inscrições confirmadas, e a elegibilidade para certificação, que exige a conclusão da atividade e a confirmação de presença. Além disso, o controle de acesso deve garantir que participantes visualizem apenas seus próprios dados, enquanto organizadores administrem somente eventos sob sua responsabilidade.

C. Atributos de Qualidade e Forças Arquiteturais

A solução prioriza escalabilidade seletiva, permitindo dimensionar serviços conforme suas demandas, e evolução independente, possibilitando alterações sem a necessidade de reimplantar toda a aplicação. Também busca garantir disponibilidade e resiliência por meio do isolamento de falhas, além de consistência nas operações de reserva e pagamento.

Complementarmente, são considerados interoperabilidade, por meio de contratos de comunicação padronizados; observabilidade, com rastreamento de requisições por identificadores de correlação; e operabilidade e segurança, com execução em ambiente conteinerizado e proteção dos endpoints por autenticação e autorização.

A partir desses requisitos, identificam-se duas forças arquiteturais centrais. A primeira é a disponibilidade e o desacoplamento, que exige consultas ágeis e notificações assíncronas, sem dependência direta de serviços auxiliares. A segunda é a consistência e o controle de concorrência, que exige alocação determinística de vagas, prevenção de *overbooking* e coordenação entre reservas e pagamentos.

Essas forças orientam a definição das fronteiras dos microsserviços, dos mecanismos de comunicação e das estratégias de controle transacional adotadas na arquitetura.

III. Projeto Arquitetural

A. Estilo Arquitetural e Decomposição dos Serviços

Adotou-se o estilo de **Arquitetura de Microsserviços Poliglota com Comunicação Híbrida**, combinando um **API Gateway** como ponto de entrada da aplicação, comunicação síncrona via REST para operações que exigem resposta imediata e mensageria assíncrona no padrão *Publish/Subscribe* para notificações e propagação de eventos. As chamadas síncronas contam com proteção por *Circuit Breaker*, visando limitar a propagação de falhas entre serviços [1], [2].

A escolha desse estilo busca permitir o escalonamento independente de componentes, especialmente do catálogo público e do serviço de inscrições, além de isolar falhas de integrações externas. A partir da análise do domínio com base em *Domain-Driven Design* (DDD) [3], foram identificadas seis macrocapacidades: identidade e controle de acesso; gestão de eventos, presença e certificação; catálogo público e descoberta; alocação de inscrições e filas; processamento financeiro; e comunicação com o usuário.

A decomposição não seguiu uma divisão individual por funcionalidade, pois isso resultaria em serviços excessivamente fragmentados e dependentes de múltiplas chamadas de rede. Em vez disso, foram considerados três critérios: afinidade transacional e coesão das regras, agrupando operações que compartilham invariantes; assimetria de carga, separando consultas públicas das operações administrativas; e isolamento de falhas e taxa de mudança, mantendo pagamentos e notificações em serviços independentes devido à integração com provedores externos.

TABELA I
Microsserviços Definidos, Contextos e Limites

Microsserviço (Stack)	Contexto Delimitado e Responsabilidades
`user-service` Java 21 / Spring (:8081)	**Identidade e Acesso:** cadastro com hash seguro, autenticação JWT e gestão de perfis. Não conhece regras de eventos.
`event-service` Java 21 / Spring (:8082)	**Gestão do Evento:** ciclo de vida de eventos e atividades, *check-in* de presença e emissão/validação de certificados.
`search-service` Python 3.12 / FastAPI (:8083)	**Catálogo Público:** consultas otimizadas *read-only* de eventos publicados, filtros e cálculo de vagas em tempo real.
`registration-service` Java 21 / Spring (:8084)	**Alocação e Filas:** solicitação e cancelamento de inscrições, lock de vagas, fila de espera automática, expiração e relatórios.
`payment-service` Python 3.12 / FastAPI (:8085)	**Faturamento:** início de cobranças junto ao provedor simulado e transições de aprovação, recusa ou expiração.
`notification-service` Python 3.12 / FastAPI (:8086)	**Comunicação:** consumo assíncrono de eventos na fila, envio de avisos, histórico do usuário e simulação de falhas.

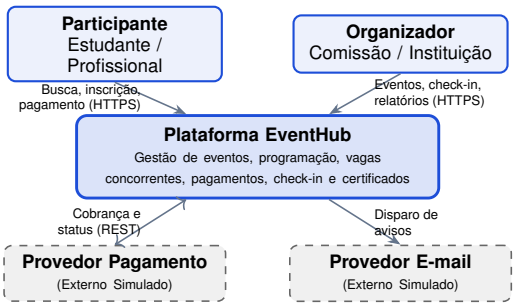


Fig. 1. Diagrama de Contexto do Sistema EventHub (C4 Model — Nível 1).

Com base nesses critérios, foram definidos seis microsserviços, cada um com responsabilidades e tecnologias específicas, conforme apresentado na Tabela I.

B. Visão Arquitetural e Comunicação entre Componentes

A visão geral da solução é apresentada por meio do modelo C4 [4]. A Fig. 1 apresenta o Diagrama de Contexto (Nível 1), identificando as interações dos organizadores e participantes com a plataforma EventHub, além das integrações com os provedores simulados de pagamento e envio de e-mails.

A Fig. 2 apresenta o Diagrama de Containers (Nível 2), detalhando os componentes internos e seus mecanismos de integração. O acesso externo ocorre exclusivamente pelo **API Gateway (Nginx)**, responsável por disponibilizar a aplicação web e encaminhar as requisições aos seis microsserviços.

Na rede interna (`eventhub-net`), os serviços utilizam chamadas REST síncronas, protegidas por *Circuit Breakers*, para operações que exigem interação direta. Para a comunicação assíncrona, utiliza-se o **RabbitMQ**, responsável pela

TABELA II
PROPRIEDADE DE ESCRITA E POLÍTICAS DE LEITURA

Escritor (dados)	Leitura por outros serviços
`user` (usuários)	Nenhuma; identidade propagada via JWT.
`event` (eventos, atividades*, presenças, certificados)	Busca (projeção do catálogo) e inscrições (validação).
`registration` (inscrições)	Busca (reservas pendentes) e eventos (check-in).
`payment` (pagamentos)	Nenhuma; estado propagado por webhook.
`notification` (notificações)	Nenhuma.

*Exceção: inscrições atualizam a ocupação sob `SELECT ... FOR UPDATE`.

distribuição de eventos e notificações. O mecanismo também conta com uma *Dead Letter Queue* (DLQ), destinada à retenção de mensagens que não puderam ser processadas.

A persistência utiliza o **PostgreSQL 16**, com propriedade lógica dos dados e segregação das operações de escrita. Para garantir a integridade das reservas sob concorrência, são utilizados mecanismos de bloqueio pessimista, como `SELECT ... FOR UPDATE`. Dessa forma, a arquitetura combina comunicação síncrona e assíncrona, buscando equilibrar consistência transacional, desacoplamento e tolerância a falhas.

IV. ESTRATÉGIA DE DADOS, COMUNICAÇÃO E DECISÕES

A. Estratégia de Persistência e Comunicação

Para viabilizar a execução reproduzível em ambiente local sem o custo de memória associado à instanciação de seis contêineres de banco de dados, adotou-se uma única instância do PostgreSQL 16, compartilhada logicamente entre os serviços. A autonomia dos limites de contexto é preservada por meio da política de **propriedade lógica estrita de escrita (*Single-Writer*)**, na qual cada tabela possui um único microsserviço responsável pelas operações de escrita e manutenção de esquema. A Tabela II apresenta a distribuição dessa responsabilidade e as políticas de acesso aos dados.

Uma particularidade dessa estratégia é o compartilhamento controlado dos dados de atividades. Embora o serviço de eventos seja responsável pelo cadastro e gerenciamento dessas informações, o serviço de inscrições possui autorização específica para atualizar a disponibilidade de vagas. Para isso, utiliza bloqueios de linha no PostgreSQL durante as operações de reserva [5], garantindo que a validação da capacidade e a atualização da ocupação ocorram atomicamente. Essa decisão evita inscrições acima da capacidade permitida sem exigir a implantação de mecanismos externos de bloqueio distribuído.

Outra decisão relevante é a utilização de uma projeção de leitura pelo serviço de busca, que calcula a disponibilidade real das atividades considerando tanto as vagas já ocupadas quanto as reservas temporárias que ainda aguardam confirmação do pagamento.

A comunicação entre os microsserviços segue um modelo híbrido, combinando chamadas síncronas HTTP/REST e mensageria assíncrona por meio do RabbitMQ. A comunicação síncrona é empregada nas operações que exigem confirmação imediata, especialmente durante o processo de inscrição e pagamento. Nesse fluxo, o serviço de inscrições solicita ao serviço de pagamentos o início de uma cobrança e recebe posteriormente o resultado por meio de uma comunicação de retorno, permitindo atualizar o estado da inscrição.

A comunicação assíncrona é utilizada principalmente para o envio de notificações, reduzindo o acoplamento entre as operações principais e o serviço responsável pelos comunicados. Os serviços de inscrições, eventos e pagamentos publicam mensagens em uma estrutura de mensageria baseada em tópicos. Nos serviços implementados com Spring Boot, a publicação ocorre somente após a confirmação da transação no banco de dados, evitando que mensagens sejam enviadas antes da efetivação das alterações.

O serviço de notificações consome essas mensagens em segundo plano. Em caso de falha no processamento, elas são encaminhadas para uma fila de mensagens não processadas, permitindo tratamento posterior. Caso o intermediador de mensagens esteja indisponível, os serviços produtores utilizam chamadas HTTP diretas como alternativa, protegidas por mecanismos de tolerância a falhas. Como medida adicional de segurança, o API Gateway bloqueia o acesso externo às operações destinadas exclusivamente à comunicação interna entre os microsserviços.

B. Consistência e Tratamento de Falhas

Para coordenar a integridade entre reservas de vagas e pagamentos sem recorrer a transações distribuídas complexas, implementou-se o padrão **Saga Orquestrada com Compensação** [2]. Conforme ilustrado na Figura 3, a criação de uma inscrição em atividade paga ocorre em duas fases: inicialmente, a vaga é reservada provisoriamente em uma transação local, confirmada antes da solicitação de cobrança. Caso o pagamento não possa ser realizado, uma operação compensatória cancela a reserva e restaura a disponibilidade da vaga.

A Figura 4 apresenta o ciclo de vida das inscrições e suas possíveis transições. Para preservar a consistência diante de falhas de rede, reenvios automáticos e operações concorrentes, são utilizados mecanismos de idempotência em diferentes etapas do fluxo. No processo de inscrição, restrições de unicidade impedem que um participante mantenha inscrições ativas duplicadas para a mesma atividade. Da mesma forma, o recebimento repetido de notificações de pagamento não provoca alterações duplicadas nos estados das inscrições ou na disponibilidade de vagas.

O mesmo princípio é aplicado ao registro de presença, que impede credenciamentos duplicados, e à emissão de certificados, que verifica previamente a existência de um documento para cada participante e atividade. Dessa forma, tentativas repetidas não produzem registros duplicados nem comprometem informações previamente emitidas.

Por fim, para prevenir *deadlocks* durante operações concorrentes de cancelamento, confirmação de pagamentos e expiração de reservas, o serviço de inscrições adota uma ordem estrita de aquisição de bloqueios no banco de dados. A atividade correspondente é sempre bloqueada antes da in-

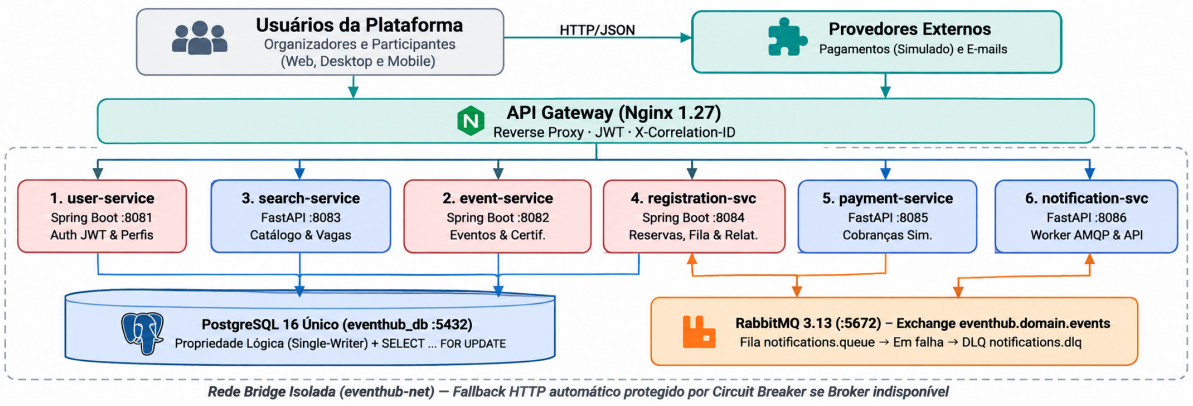


Fig. 2. Diagrama de Containers e Fluxos Norte–Sul e Leste–Oeste do EventHub (C4 Model — Nível 2).

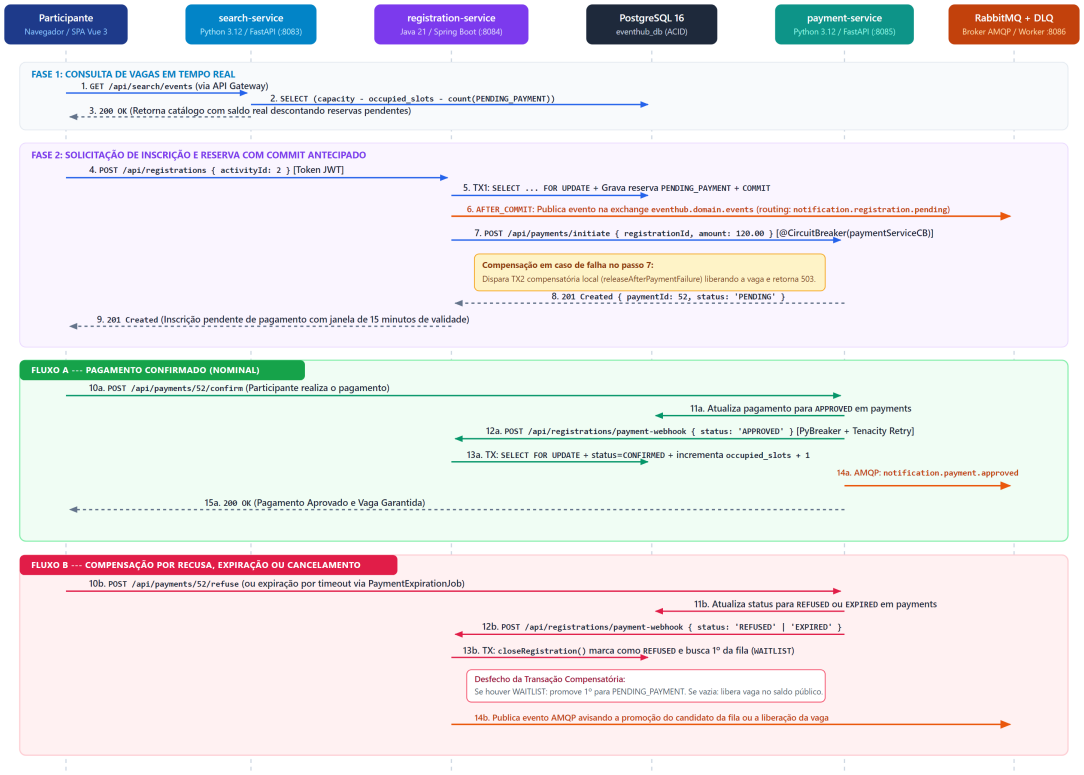


Fig. 3. Diagrama de Sequência da Inscrição em Atividade Paga e Compensações da Saga.

scrição, reduzindo a possibilidade de conflitos entre transações simultâneas.

C. Decisões Arquiteturais (ADRs)

As principais decisões estruturantes do projeto foram consolidadas em quatro registros de decisão arquitetural (ADRs):

- **ADR-01 (Comunicação Híbrida e Mensageria Assíncrona):** centralizou o tráfego externo no Nginx, adotou chamadas REST síncronas protegidas por *Circuit Breaker* para o fluxo de checkout e mensageria no RabbitMQ com retenção em Dead Letter Queue (`notifications.dlq`) [6]. *Trade-off:* resposta imediata para o usuário e resiliência

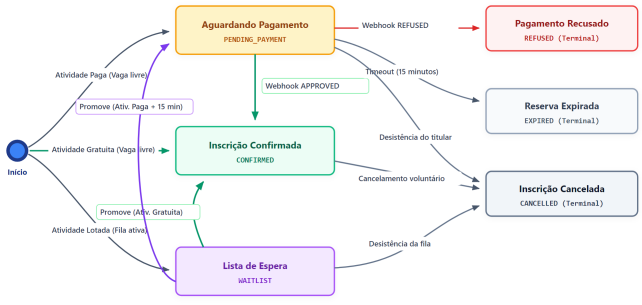


Fig. 4. Ciclo de vida e transições de estado das inscrições.

a falhas parciais, com o custo operacional de manter um broker;

- **ADR-02 (PostgreSQL Único com Propriedade Lógica Estrita):** estabeleceu uma única instância compartilhada do PostgreSQL com responsabilidade exclusiva de escrita por tabela (*Single-Writer*), em substituição a instâncias físicas separadas. *Trade-off:* reduz drasticamente o consumo de memória em desenvolvimento e viabiliza projeções de catálogo em tempo real, exigindo rigor no controle de esquema;
- **ADR-03 (Bloqueio Pessimista e Publicação Pós-Commit):** adotou bloqueios de linha `FOR UPDATE` no PostgreSQL para validação atômica de vagas e postergou o disparo de eventos para após a efetivação da transação (*after commit*). *Trade-off:* elimina comprovadamente o *overbooking* e evita o descarte de requisições legítimas, introduzindo breve serialização sob alta concorrência na mesma atividade;
- **ADR-04 (Arquitetura Poliglota Java e Python):** distribuiu os serviços entre Java 21 / Spring Boot 3 (para operações de alta densidade transacional e regras de negócio) e Python 3.12 / FastAPI (para catálogo de leitura assíncrona, pagamentos e consumo de eventos). *Trade-off:* adequação técnica ótima ao perfil de carga de cada capacidade, com o custo de manter dois ecossistemas de desenvolvimento.

V. Implementação e Infraestrutura

A. Ecossistemas Tecnológicos e Organização Interna

A implementação do EventHub adota uma abordagem poliglota pragmática, distribuindo as responsabilidades entre dois ecossistemas tecnológicos. Java 21 com Spring Boot 3 é utilizado nos serviços de maior densidade transacional e responsabilidade sobre as regras de negócio (`user-service`, `event-service` e `registration-service`). Essa escolha fundamenta-se na maturidade do ecossistema para persistência com Spring Data JPA e Hibernate, gerenciamento transacional declarativo por meio de `@Transactional` e implementação de bloqueios pessimistas no PostgreSQL. Além disso, a integração com o Resilience4j oferece mecanismos de tolerância a falhas, enquanto a tipagem estática contribui para a segurança e a consistência das operações de domínio.

Por sua vez, Python 3.12 com FastAPI é empregado nos serviços voltados à comunicação assíncrona, consultas ao catálogo e integração com sistemas externos (`search-service`, `payment-service` e `notification-service`). Sua arquitetura baseada em ASGI, executada com Uvicorn e Starlette, permite lidar com operações de entrada e saída de forma não bloqueante. O Pydantic v2 é utilizado na validação e serialização dos dados, enquanto bibliotecas como `aio_pika`, PyBreaker e Tenacity apoiam, respectivamente, a comunicação com o broker, a implementação de disjuntores de circuito e as políticas de retentativa.

Independentemente da linguagem adotada, os microsserviços seguem uma organização interna inspirada nos princípios da Arquitetura Limpa e do Design Orientado a Domínio (DDD) [3], [7]. Essa organização separa as responsabilidades de apresentação, aplicação, domínio e acesso a dados. A camada de apresentação recebe as requisições HTTP, valida os dados de entrada por meio de DTOs e esquemas Pydantic e converte exceções em respostas HTTP adequadas, seguindo o padrão *Problem Details* [8]. A camada de aplicação e domínio concentra os casos de uso, as regras de negócio e o gerenciamento das operações transacionais, enquanto a camada de acesso a dados encapsula as consultas e operações de persistência no PostgreSQL. Os modelos e objetos de transferência, por sua vez, distinguem as estruturas utilizadas internamente daquelas expostas nas interfaces de comunicação, evitando o acoplamento entre o modelo persistente e os contratos externos.

B. Segurança, Autenticação e Controle de Acesso

A segurança do EventHub utiliza um modelo de autenticação sem estado (*stateless*), baseado em JSON Web Tokens (JWT) [9]. Essa abordagem elimina a necessidade de manter sessões nos servidores, facilitando a escalabilidade horizontal dos microsserviços.

O processo de autenticação é centralizado no serviço de identidade, responsável por validar as credenciais dos usuários e verificar os hashes das senhas por meio do algoritmo BCrypt. Após uma autenticação bem-sucedida, é emitido um token JWT assinado com HMAC-SHA256, utilizando uma chave secreta compartilhada pela infraestrutura. O token contém informações sobre o usuário, seu endereço de e-mail, seu papel na plataforma e seu prazo de validade, configurado em 24 horas. Nas requisições seguintes, o cliente encaminha essa credencial no cabeçalho HTTP de autorização, permitindo a identificação do usuário nos demais serviços.

A validação dos tokens ocorre de maneira descentralizada: cada microsserviço verifica autonomamente a assinatura e a expiração da credencial, utilizando Spring Security nas aplicações Java ou mecanismos de dependência do FastAPI nas aplicações Python. Dessa forma, não é necessário consultar o serviço de identidade a cada requisição protegida, reduzindo a dependência de comunicação síncrona e evitando a criação de um ponto adicional de contenção.

O controle de acesso segue o modelo baseado em papéis (RBAC), restringindo as operações conforme o perfil associado ao token. Organizadores possuem permissões para criar e modificar eventos, configurar lotes de vagas, registrar presenças e solicitar a emissão de certificados. Participantes, por sua vez, podem consultar o catálogo, solicitar inscrições, realizar pagamentos, cancelar suas próprias reservas e acessar o histórico de certificados e comprovantes. Requisições que tentam executar operações sem a autorização necessária são rejeitadas com o código HTTP 403 (Forbidden).

C. Orquestração e Infraestrutura com Docker

A infraestrutura do EventHub foi conteinerizada e parametrizada por meio do Docker Compose, permitindo a execução coordenada e reprodutível dos componentes em diferentes ambientes hospedeiros [10]. O ambiente é composto por nove containers conectados à rede interna `eventhub-net`, configurada com o driver *bridge*. Essa rede proporciona isolamento do tráfego entre os componentes e resolução automática de nomes por meio do DNS interno do Docker. O ponto de entrada da aplicação é o `eventhub-gateway`, baseado em Nginx 1.27-alpine e disponibilizado na porta 8080 do hospedeiro. Além de entregar os arquivos estáticos da interface web, desenvolvida com Vue 3, o gateway atua como proxy reverso, encaminhando as requisições aos respectivos microsserviços.

Os seis microsserviços são executados em containers independentes, utilizando OpenJDK 21 Eclipse Temurin para as aplicações Spring Boot e Python 3.12-slim com Uvicorn para os serviços FastAPI. A camada de persistência é composta pelo PostgreSQL 16-alpine, responsável por hospedar a base relacional compartilhada. A durabilidade dos dados é assegurada por um volume persistente, enquanto um script de inicialização realiza a criação das estruturas necessárias e a inserção dos dados iniciais. A comunicação assíncrona entre os serviços é suportada pelo RabbitMQ 3.13-management-alpine, que disponibiliza o protocolo AMQP 0-9-1, filas duráveis e uma fila de mensagens não processadas (Dead Letter Queue), utilizada no tratamento de falhas durante o processamento das notificações.

Por fim, a inicialização dos componentes é coordenada por dependências declaradas no Docker Compose e verificações de integridade (*healthchecks*). O PostgreSQL utiliza o comando `pg_isready` para verificar sua disponibilidade, enquanto o RabbitMQ executa o diagnóstico nativo `rabbitmq-diagnostics ping`. Os microsserviços de aplicação aguardam que esses componentes estejam operacionais antes de iniciar, reduzindo falhas decorrentes de tentativas prematuras de conexão durante a inicialização do ambiente.

VI. Validação Experimental e Resultados

A. Metodologia e Testes de Concorrência

A validação do EventHub foi estruturada em uma abordagem multinível, contemplando testes unitários, testes de integração entre serviços, verificações de segurança baseadas em papéis (RBAC), ensaios automatizados de concorrência e injeção controlada de falhas. Ao todo, foram formulados e executados 39 casos de teste, distribuídos entre os componentes da solução: seis no serviço de identidade, onze no serviço de eventos e atividades, seis no catálogo de busca, seis no serviço de inscrições e relatórios, cinco no módulo financeiro, cinco no serviço de notificações, dois na infraestrutura e quatro no laboratório de caos. As verificações combinaram a utilização da interface web, da documentação executável OpenAPI/Swagger e do módulo automatizado de validação arquitetural (*Chaos Lab*).

Entre os ensaios realizados, destaca-se o teste de concorrência do mecanismo de alocação de vagas, cujo objetivo foi verificar a integridade das inscrições diante de solicitações simultâneas para uma atividade com capacidade limitada (C). Para isso, foram provisionados participantes com credenciais e tokens JWT individuais, cujas requisições de inscrição foram disparadas concorrentemente por meio de um executor automatizado, aumentando a contenção sobre a camada de persistência.

Em um ensaio com capacidade $C = 5$ e $N = 25$ solicitações concorrentes disparadas simultaneamente, a rajada completa foi processada em 1245,89 ms, atingindo uma vazão de 20,07 requisições por segundo e latência média de 1015,35 ms (com mínima de 895,54 ms e máxima de 1217,13 ms). Registraram-se rigorosamente cinco inscrições confirmadas (`CONFIRMED`), enquanto as vinte solicitações excedentes foram direcionadas à lista de espera (`WAITLIST`) em ordem FIFO estrita. Não foram observados erros internos de servidor (HTTP 500), quedas de conexão ou inconsistências na base de dados. Esse resultado comprova empiricamente a eficácia do bloqueio pessimista relacional (`SELECT ... FOR UPDATE`), que serializa com segurança as transações concorrentes na linha da atividade, eliminando condições de corrida durante a consulta e a reserva atômica de vagas.

Além disso, consultas aos indicadores gerenciais e ao catálogo público confirmaram a consistência entre o número de participantes inscritos, a quantidade de vagas restantes e os registros persistidos no banco de dados. Dessa forma, o experimento demonstrou a capacidade do sistema de preservar a integridade das operações mesmo sob elevada concorrência, sem ocorrência de sobreinscrição (*overbooking*).

B. Resiliência e Consistência dos Fluxos de Negócio

A resiliência operacional foi avaliada por meio de ensaios controlados de injeção de falhas, considerando principalmente a indisponibilidade de componentes e a necessidade de compensação de operações distribuídas. No primeiro cenário, foi simulada uma falha no serviço de notificações, fazendo com que suas chamadas retornassem HTTP 503 (Service Unavailable). Mesmo diante dessa indisponibilidade, os fluxos de inscrição e confirmação de pagamento foram concluídos normalmente. Esse comportamento decorre do desacoplamento proporcionado pela mensageria assíncrona: as notificações são publicadas após a confirmação da transação principal (*after commit*) e processadas pelo RabbitMQ, evitando que a indisponibilidade do serviço de notificações comprometa as operações essenciais da plataforma. As mensagens permaneceram preservadas para processamento posterior, incluindo o tratamento de falhas por meio da Dead Letter Queue, até o restabelecimento do consumidor.

Outro cenário avaliou a compensação de operações em uma Saga envolvendo a recusa de pagamento. Em uma atividade paga com apenas uma vaga disponível, a inscrição do primeiro participante gerou uma reserva pendente de pagamento, com prazo de 15 minutos. Um segundo participante foi direcionado à lista de espera. Após a simulação da recusa do pagamento

inicial, a inscrição correspondente foi marcada como recusada, a reserva foi liberada e o primeiro participante da fila foi promovido automaticamente para uma nova reserva pendente, também com prazo de 15 minutos. Com a aprovação do pagamento subsequente, a inscrição passou ao estado confirmado. O experimento demonstrou a execução adequada das ações compensatórias e a preservação da disponibilidade das vagas durante as transições do fluxo financeiro.

C. Segurança, Regras de Negócio e Síntese dos Resultados

A validação das políticas de segurança e das restrições de negócio foi realizada por meio de testes negativos, nos quais foram executadas operações sem as permissões ou condições necessárias. No controle de acesso entre participantes, tentativas de consultar ou cancelar inscrições pertencentes a outros usuários foram rejeitadas com HTTP 403 (Forbidden), garantindo o isolamento dos dados. O mesmo código foi retornado para solicitações de acesso aos relatórios gerenciais realizadas por participantes ou por organizadores que não eram responsáveis pelo evento.

Também foram verificadas restrições relacionadas à integridade das atividades. Tentativas de reduzir a capacidade de uma atividade para um valor inferior ao número de participantes já confirmados foram rejeitadas com HTTP 422 (Unprocessable Entity). De maneira semelhante, solicitações de emissão de certificados para atividades não concluídas ou para participantes sem registro de presença (*check-in*) foram bloqueadas, impedindo a geração de documentos sem o cumprimento dos requisitos estabelecidos.

De forma geral, os resultados experimentais demonstraram conformidade em todos os cenários avaliados. Os testes de concorrência confirmaram a alocação correta das vagas, sem ocorrência de sobreinscrição; os ensaios de falha evidenciaram a independência entre os fluxos principais e o serviço de notificações, além da compensação adequada em recusas de pagamento; e as verificações negativas comprovaram a aplicação das políticas de acesso e das restrições de negócio. Assim, os experimentos sustentam a eficácia dos mecanismos de consistência, resiliência e segurança implementados no EventHub.

VII. Discussão Crítica e Conclusão

A. Análise de Ganhos e Custos da Arquitetura

A concepção arquitetural de um sistema distribuído exige a ponderação entre os benefícios obtidos e os custos operacionais envolvidos [1], [11]. No EventHub, as decisões arquiteturais proporcionaram ganhos relacionados ao desacoplamento, à resiliência, à integridade transacional e à flexibilidade tecnológica, mas também introduziram desafios decorrentes da distribuição dos componentes.

Entre os principais benefícios, destaca-se o desacoplamento temporal proporcionado pela mensageria assíncrona. A separação entre as operações transacionais essenciais e o processamento de notificações permite que a indisponibilidade de serviços periféricos não comprometa diretamente os fluxos de inscrição e pagamento. A adoção de uma arquitetura poliglota também possibilitou distribuir as tecnologias de acordo com as características de cada serviço, utilizando Python com FastAPI nas operações de consulta e integração assíncrona, enquanto Java com Spring Boot foi empregado nos componentes com maior concentração de regras transacionais. Outro ganho importante foi a preservação da disponibilidade de vagas por meio de bloqueios pessimistas no PostgreSQL, associados ao controle de reservas pendentes. Os testes de concorrência demonstraram a eficácia desse mecanismo no cenário avaliado, sem ocorrência de sobreinscrição (*overbooking*). Por fim, a autenticação baseada em JWT e a validação descentralizada de permissões reduziram a dependência de consultas síncronas ao serviço de identidade, favorecendo a escalabilidade dos componentes.

Em contrapartida, a distribuição das responsabilidades entre seis microsserviços, implementados em duas linguagens e ecossistemas distintos, aumentou a complexidade de implantação, configuração e depuração. O gerenciamento das redes, dependências e ciclos de vida dos containers exige automação e padronização, enquanto o rastreamento de falhas distribuídas é mais complexo do que em uma aplicação monolítica. Além disso, embora os bloqueios pessimistas preservem a integridade das operações concorrentes, eles introduzem contenção quando múltiplas transações disputam o mesmo recurso, tornando necessário controlar a duração das transações. Por fim, a utilização de uma instância compartilhada do PostgreSQL, embora adequada às restrições de infraestrutura do protótipo, exige disciplina na separação lógica das responsabilidades de persistência e não proporciona o mesmo isolamento de uma abordagem com bancos de dados independentes por serviço.

B. Limitações do Protótipo e Evoluções Futuras

Apesar dos resultados obtidos, o protótipo apresenta limitações que devem ser consideradas em uma eventual implantação em larga escala. A infraestrutura atual utiliza Docker Compose em um único ambiente hospedeiro, solução adequada ao desenvolvimento, à homologação e à execução dos experimentos, mas limitada em termos de alta disponibilidade e escalabilidade horizontal entre diferentes máquinas. Como evolução, recomenda-se a adoção de uma plataforma de orquestração, como Kubernetes, permitindo balanceamento de carga, escalonamento automático de réplicas e maior tolerância a falhas de infraestrutura.

Outra possibilidade de aprimoramento envolve a publicação de eventos de domínio. Atualmente, o mecanismo utiliza um processamento pós-commit para encaminhar mensagens ao RabbitMQ. Entretanto, uma falha da aplicação após a confirmação da transação no banco de dados e antes da publicação da mensagem pode ocasionar a perda do evento. Para mitigar esse risco, recomenda-se a adoção do padrão *Transactional Outbox*, associado a mecanismos de *Change Data Capture* (CDC), como Debezium. Essa abordagem permite registrar os eventos na mesma transação dos dados de negócio e encaminhá-los posteriormente ao broker, oferecendo maior confiabilidade na publicação e suporte ao modelo de entrega *at-least-once*.

No âmbito da persistência, a utilização de uma instância compartilhada do PostgreSQL pode evoluir para uma segregação física das bases de dados, seguindo o princípio *Database-per-Service*. Essa mudança ampliaria o isolamento entre os microsserviços, embora introduza desafios adicionais de consistência e integração entre dados. Complementarmente, a adoção de estratégias de segregação entre leitura e escrita, como o padrão CQRS, poderia otimizar as consultas do catálogo público por meio de estruturas de leitura específicas.

Por fim, a observabilidade constitui outra oportunidade de evolução. A incorporação de ferramentas como OpenTelemetry, Jaeger e Prometheus permitiria acompanhar métricas, rastrear requisições distribuídas e analisar o comportamento dos componentes de forma integrada. Essa capacidade seria especialmente relevante para identificar gargalos de desempenho, investigar falhas em operações assíncronas e acompanhar a saúde da infraestrutura em ambientes de produção.

C. Considerações Finais

O desenvolvimento do EventHub demonstrou a viabilidade prática de uma arquitetura de microsserviços orientada ao domínio (DDD) para a gestão de eventos acadêmicos e profissionais. A solução integrou serviços especializados para gerenciamento de usuários, eventos, inscrições, pagamentos, consultas e notificações, utilizando mecanismos de comunicação síncrona e assíncrona para coordenar suas operações.

Os resultados experimentais evidenciaram a eficácia das decisões arquiteturais adotadas nos cenários avaliados, especialmente na preservação da integridade das inscrições sob concorrência, no tratamento de falhas parciais e na aplicação das políticas de segurança e das regras de negócio. A implementação de mecanismos de bloqueio, compensação de operações e processamento assíncrono contribuiu para a confiabilidade dos principais fluxos da plataforma.

Dessa forma, o projeto demonstra que a aplicação criteriosa de padrões consolidados de sistemas distribuídos permite construir soluções modulares, com responsabilidades bem delimitadas e mecanismos de resiliência. Embora existam limitações relacionadas à infraestrutura, à persistência compartilhada e à observabilidade, os resultados alcançados fornecem uma base consistente para a evolução do EventHub em direção a um ambiente de produção com maior escala e disponibilidade.

Declaração de Uso de Ferramentas de Inteligência Artificial

Em conformidade com as diretrizes de integridade e transparência técnica, declara-se que modelos de linguagem de inteligência artificial foram utilizados como ferramentas auxiliares de produtividade durante o desenvolvimento do projeto. Sua utilização envolveu o apoio à escrita e à refatoração de trechos de código, à geração de scripts de teste e à elaboração de diagramas, além do auxílio na redação e padronização tipográfica deste relatório no formato IEEEtran. As decisões arquiteturais, regras de negócio, validações experimentais e interpretações de engenharia foram revisadas e verificadas pelos autores.

REFERENCES

[1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[2] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018.

[3] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003.

[4] S. Brown, "The C4 model for visualising software architecture," https://c4model.com/, 2018.

[5] PostgreSQL Global Development Group, "PostgreSQL 16 documentation: Explicit locking," https://www.postgresql.org/docs/16/explicit-locking.html, 2024.

[6] Broadcom Inc., "RabbitMQ documentation: Dead letter exchanges," https://www.rabbitmq.com/docs/dlx, 2024.

[7] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.

[8] M. Nottingham and E. Wilde, "Problem details for HTTP APIs," Internet Engineering Task Force (IETF), RFC 7807, 2016. [Online]. Available: https://www.rfc-editor.org/info/rfc7807

[9] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), RFC 7519, 2015. [Online]. Available: https://www.rfc-editor.org/info/rfc7519

[10] Docker Inc., "Docker compose documentation," https://docs.docker.com/compose/, 2024.

[11] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.